

SQL Cheat Sheet

SQL estándar con notas de Oracle, PostgreSQL y MySQL donde difieren.

Orden de escritura vs orden de ejecución

```
SELECT  columnas      -- 5
FROM    tabla         -- 1
JOIN    otra ON ...   -- 2
WHERE   filtro        -- 3 (filtra FILAS, antes de agrupar)
GROUP BY columnas     -- 4
HAVING  filtro_grupo  -- 6 (filtra GRUPOS, después de agrupar)
ORDER BY columnas     -- 7
LIMIT / FETCH         -- 8
```

Por eso no podés usar un alias del `SELECT` en el `WHERE`, pero sí en el `ORDER BY`.

SELECT básico

```
SELECT DISTINCT c.id, c.nombre AS cliente
FROM   cliente c
WHERE  c.estado = 'ACTIVO'
      AND c.fecha_alta >= DATE '2026-01-01'
      AND c.sucursal IN ('CBA', 'BUE')
      AND c.nombre LIKE 'GON%'           -- % cualquier cadena, _ un carácter
      AND c.saldo BETWEEN 1000 AND 5000 -- inclusive
      AND c.baja IS NULL                -- nunca "= NULL"
ORDER BY c.fecha_alta DESC NULLS LAST
FETCH FIRST 20 ROWS ONLY;               -- estándar / Oracle 12c+
-- LIMIT 20 OFFSET 40;                  -- PostgreSQL / MySQL
```

NULL: las tres reglas

- Cualquier comparación con `NULL` da desconocido → usá `IS NULL` / `IS NOT NULL`.
- Los agregados **ignoran** `NULL` (`COUNT(col) ≠ COUNT(*)`).
- `NULL + 5 = NULL`, `'a' || NULL = NULL` (en Oracle la concatenación trata `' '` como `NULL`).

```
COALESCE(telefono, celular, 'sin dato') -- primer no nulo (estándar)
NULLIF(a, b)                             -- NULL si a = b
NVL(x, 0), NVL2(x, si, no)                -- Oracle
IFNULL(x, 0)                             -- MySQL
```

JOINS

```
FROM   pedido p
JOIN   cliente c ON c.id = p.cliente_id -- INNER: solo lo que matchea
LEFT JOIN envio e ON e.pedido_id = p.id -- todo pedido, tenga envío o no
RIGHT JOIN ...                          -- al revés (poco usado)
FULL OUTER JOIN ...                     -- todo de ambos lados
CROSS JOIN ...                          -- producto cartesiano
```

```
-- Anti-join: pedidos SIN envío
SELECT p.* FROM pedido p
LEFT JOIN envio e ON e.pedido_id = p.id
WHERE e.id IS NULL;
```

```
-- Self-join: empleado y su jefe
SELECT e.nombre, j.nombre AS jefe
FROM   empleado e
LEFT JOIN empleado j ON j.id = e.jefe_id;
```

En un `LEFT JOIN`, poner la condición de la tabla derecha en el `WHERE` lo convierte de hecho en un `INNER JOIN`. Si querés filtrar sin perder filas, va en el `ON`.

Agregaciones

```

SELECT  c.sucursal,
        COUNT(*)           AS cantidad,
        COUNT(c.email)     AS con_email,    -- ignora NULLs
        SUM(c.saldo)       AS total,
        AVG(c.saldo)       AS promedio,
        MIN(c.fecha_alta)  AS primero,
        MAX(c.saldo)       AS mayor
FROM    cliente c
WHERE   c.estado = 'ACTIVO'
GROUP BY c.sucursal
HAVING  COUNT(*) > 10
ORDER BY total DESC;

```

Toda columna del **SELECT** que no sea agregada debe estar en el **GROUP BY**.

Subconsultas y CTEs

```

-- Escalar
SELECT nombre, (SELECT COUNT(*) FROM pedido p WHERE p.cliente_id = c.id) AS pedidos FROM cliente c;

-- IN / EXISTS (EXISTS suele ser mejor con muchos datos y es seguro frente a NULLs)
SELECT * FROM cliente c WHERE EXISTS (SELECT 1 FROM pedido p WHERE p.cliente_id = c.id);
SELECT * FROM cliente c WHERE c.id NOT IN (SELECT cliente_id FROM baja WHERE cliente_id IS NOT NULL);

-- CTE: legible y reutilizable dentro de la misma query
WITH ventas_mes AS (
    SELECT cliente_id, SUM(importe) AS total
    FROM    pedido
    WHERE   fecha >= DATE '2026-01-01'
    GROUP BY cliente_id
), top AS (
    SELECT * FROM ventas_mes WHERE total > 100000
)
SELECT c.nombre, t.total
FROM    top t JOIN cliente c ON c.id = t.cliente_id;

-- CTE recursiva: jerarquías
WITH RECURSIVE arbol (id, nombre, jefe_id, nivel) AS (
    SELECT id, nombre, jefe_id, 1 FROM empleado WHERE jefe_id IS NULL
    UNION ALL
    SELECT e.id, e.nombre, e.jefe_id, a.nivel + 1
    FROM    empleado e JOIN arbol a ON e.jefe_id = a.id
)
SELECT * FROM arbol ORDER BY nivel;
-- Oracle: WITH arbol (...) AS (...) sin la palabra RECURSIVE

```

Window functions

```

SELECT p.id, p.cliente_id, p.importe,
       ROW_NUMBER() OVER (PARTITION BY p.cliente_id ORDER BY p.fecha DESC) AS nro,
       RANK()        OVER (ORDER BY p.importe DESC) AS ranking,    -- deja huecos
       DENSE_RANK() OVER (ORDER BY p.importe DESC) AS ranking2,    -- sin huecos
       SUM(p.importe) OVER (PARTITION BY p.cliente_id
                           SUM(p.importe) OVER (ORDER BY p.fecha
                                                 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS acumulado,
       LAG(p.importe) OVER (ORDER BY p.fecha) AS anterior,
       LEAD(p.importe) OVER (ORDER BY p.fecha) AS siguiente,
       AVG(p.importe) OVER (ORDER BY p.fecha ROWS 2 PRECEDING) AS media_movil
FROM    pedido p;

-- Patrón clásico: el último registro por cliente
SELECT * FROM (
    SELECT p.*, ROW_NUMBER() OVER (PARTITION BY cliente_id ORDER BY fecha DESC) rn
    FROM    pedido p
) WHERE rn = 1;

```

A diferencia de **GROUP BY**, las window functions **no colapsan filas**: agregan una columna calculada a cada fila.

Conjuntos

```

SELECT id FROM a UNION      SELECT id FROM b; -- une y elimina duplicados (más lento)
SELECT id FROM a UNION ALL  SELECT id FROM b; -- une sin deduplicar (preferido si no hace falta)
SELECT id FROM a INTERSECT SELECT id FROM b;
SELECT id FROM a EXCEPT   SELECT id FROM b; -- Oracle: MINUS

```

CASE y funciones

```

SELECT nombre,
       CASE WHEN saldo < 0      THEN 'DEUDOR'
            WHEN saldo < 10000 THEN 'ESTANDAR'
            ELSE 'PREMIUM' END AS segmento,
       CASE estado WHEN 'A' THEN 'Activo' WHEN 'B' THEN 'Baja' ELSE '?' END AS desc_estado,
       -- agregación condicional (pivot manual)
       SUM(CASE WHEN tipo = 'CR' THEN importe ELSE 0 END) AS creditos
FROM cliente;

```

Categoría	Funciones
Texto	UPPER, LOWER, TRIM, LPAD, SUBSTR, LENGTH, REPLACE, INSTR, CONCAT / \ \
Números	ROUND, TRUNC, CEIL, FLOOR, ABS, MOD, POWER
Fecha	CURRENT_DATE, SYSDATE (Ora), NOW() (PG/MySQL), EXTRACT(YEAR FROM f), ADD_MONTHS, TRUNC(f, 'MM'), MONTHS_BETWEEN
Conversión	CAST(x AS NUMBER), TO_CHAR(f, 'DD/MM/YYYY'), TO_DATE, TO_NUMBER

DML

```

INSERT INTO cliente (id, nombre, estado) VALUES (seq_cliente.NEXTVAL, 'Gonzalez', 'ACTIVO');
INSERT INTO cliente (id, nombre) SELECT id, nombre FROM cliente_stage;

UPDATE cliente SET estado = 'BAJA', fecha_baja = SYSDATE WHERE id = 10;

UPDATE cliente c                                -- update con subconsulta
  SET c.saldo = (SELECT SUM(m.importe) FROM movimiento m WHERE m.cliente_id = c.id)
 WHERE EXISTS (SELECT 1 FROM movimiento m WHERE m.cliente_id = c.id);

DELETE FROM cliente WHERE fecha_baja < DATE '2020-01-01';
TRUNCATE TABLE staging;                        -- DDL: vacía rápido, no se puede rollbackear, no dispara triggers

-- UPSERT
MERGE INTO cliente c
USING (SELECT :id AS id, :nombre AS nombre FROM dual) src
  ON (c.id = src.id)
 WHEN MATCHED THEN UPDATE SET c.nombre = src.nombre
 WHEN NOT MATCHED THEN INSERT (id, nombre) VALUES (src.id, src.nombre);
-- PostgreSQL: INSERT ... ON CONFLICT (id) DO UPDATE SET nombre = EXCLUDED.nombre;
-- MySQL:      INSERT ... ON DUPLICATE KEY UPDATE nombre = VALUES(nombre);

```

Siempre probá un `UPDATE` / `DELETE` primero como `SELECT` con el mismo `WHERE`. Y ojo con el autocommit.

DDL y constraints

```

CREATE TABLE cuenta (
  id          NUMBER(19)      GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  cliente_id  NUMBER(19)      NOT NULL,
  numero      VARCHAR2(22)    NOT NULL,
  saldo       NUMBER(15,2)    DEFAULT 0 NOT NULL,
  tipo        VARCHAR2(2)     NOT NULL,
  creado      TIMESTAMP       DEFAULT SYSTIMESTAMP,
  CONSTRAINT uk_cuenta_numero UNIQUE (numero),
  CONSTRAINT fk_cuenta_cliente FOREIGN KEY (cliente_id) REFERENCES cliente (id),
  CONSTRAINT ck_cuenta_tipo CHECK (tipo IN ('CA','CC'))
);

ALTER TABLE cuenta ADD (moneda VARCHAR2(3) DEFAULT 'ARS');
ALTER TABLE cuenta MODIFY (numero VARCHAR2(34));
ALTER TABLE cuenta DROP COLUMN moneda;
ALTER TABLE cuenta RENAME COLUMN creado TO fecha_creacion;
DROP TABLE cuenta CASCADE CONSTRAINTS;

CREATE INDEX idx_cuenta_cliente ON cuenta (cliente_id);
CREATE UNIQUE INDEX uk_x ON cuenta (numero, moneda);
CREATE OR REPLACE VIEW v_cuenta_activa AS SELECT * FROM cuenta WHERE estado = 'A';

```

Transacciones

```

BEGIN;                                -- Oracle: implícito con el primer DML
UPDATE cuenta SET saldo = saldo - 100 WHERE id = 1;
SAVEPOINT antes_credito;
UPDATE cuenta SET saldo = saldo + 100 WHERE id = 2;
ROLLBACK TO antes_credito;
COMMIT;                                -- o ROLLBACK

SELECT * FROM cuenta WHERE id = 1 FOR UPDATE;    -- bloquea la fila
SELECT ... FOR UPDATE NOWAIT;                    -- falla en vez de esperar
SELECT ... FOR UPDATE SKIP LOCKED;                -- útil para colas de trabajo

```

Nivel de aislamiento	Evita
READ UNCOMMITTED	nada (lecturas sucias)
READ COMMITTED (default Oracle/PG)	lecturas sucias
REPEATABLE READ (default MySQL)	+ lecturas no repetibles
SERIALIZABLE	+ phantom reads

Performance — lo mínimo que hay que saber

- Indexá las columnas de **JOIN**, **WHERE** y **ORDER BY** de alta cardinalidad; la PK ya viene indexada.
- **Matás el índice** si aplicás una función a la columna: **WHERE UPPER(nombre) = 'X'** o **WHERE TRUNC(fecha) = ...**. Usá rangos: **WHERE fecha >= :d AND fecha < :d + 1** (o un índice basado en función).
- **LIKE '%texto'** (comodín al inicio) no usa índice.
- Evitá **SELECT *** en producción: trae columnas de más y rompe si cambia la tabla.
- **EXISTS** suele ganarle a **IN** con subconsultas grandes; **UNION ALL** a **UNION**.
- Índice compuesto: el orden importa. **(cliente_id, fecha)** sirve para filtrar por **cliente_id** solo, pero no por **fecha** sola.
- Mirá el plan: **EXPLAIN PLAN FOR ... + SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY)** (Oracle) o **EXPLAIN ANALYZE ...** (PostgreSQL). Buscá FULL TABLE SCAN sobre tablas grandes.
- Usá **bind variables** (**:param**, **?**), nunca concatenación de strings: evita SQL injection y reutiliza el plan.

Específico de Oracle

```

SELECT SYSDATE FROM dual;                -- tabla dummy obligatoria
SELECT seq_cliente.NEXTVAL FROM dual;

-- Paginación clásica (antes de 12c)
SELECT * FROM (SELECT a.*, ROWNUM rn FROM (SELECT ... ORDER BY fecha) a WHERE ROWNUM <= 40)
WHERE rn > 20;

DECODE(estado, 'A', 'Activo', 'B', 'Baja', 'Otro')    -- CASE abreviado, propio de Oracle
LISTAGG(nombre, ', ' ) WITHIN GROUP (ORDER BY nombre) -- concatenar filas en una cadena
TO_CHAR(fecha, 'DD/MM/YYYY HH24:MI:SS')
TRUNC(SYSDATE, 'MM')                                -- primer día del mes
ADD_MONTHS(SYSDATE, -1)
CONNECT BY PRIOR id = jefe_id START WITH jefe_id IS NULL -- jerarquías estilo Oracle

```

Procedimientos almacenados (PL/SQL)

```

CREATE OR REPLACE PROCEDURE sp_acreditar (
  p_cuenta_id IN NUMBER,
  p_importe   IN NUMBER,
  p_resultado OUT VARCHAR2
) IS
  v_saldo NUMBER(15,2);
  e_cuenta_invalida EXCEPTION;
BEGIN
  SELECT saldo INTO v_saldo FROM cuenta WHERE id = p_cuenta_id FOR UPDATE;

  IF p_importe <= 0 THEN
    RAISE e_cuenta_invalida;
  END IF;

  UPDATE cuenta SET saldo = saldo + p_importe WHERE id = p_cuenta_id;
  INSERT INTO movimiento (cuenta_id, importe, fecha) VALUES (p_cuenta_id, p_importe, SYSDATE);

  p_resultado := 'OK';
  COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN p_resultado := 'CUENTA_INEXISTENTE'; ROLLBACK;
  WHEN e_cuenta_invalida THEN p_resultado := 'IMPORTE_INVALIDO'; ROLLBACK;
  WHEN OTHERS THEN p_resultado := SQLERRM; ROLLBACK; RAISE;
END sp_acreditar;
/

```

```

-- Function
CREATE OR REPLACE FUNCTION fn_saldo (p_id NUMBER) RETURN NUMBER IS
v NUMBER;
BEGIN
    SELECT saldo INTO v FROM cuenta WHERE id = p_id;
    RETURN v;
END;
/

-- Cursor explícito + bulk
FOR r IN (SELECT id, saldo FROM cuenta WHERE estado = 'A') LOOP
    ...
END LOOP;

SELECT id BULK COLLECT INTO v_ids FROM cuenta;
FORALL i IN v_ids.FIRST .. v_ids.LAST
    UPDATE cuenta SET revisada = 'S' WHERE id = v_ids(i);

```

Lamarlo desde Java:

```

// JDBC puro
try (CallableStatement cs = conn.prepareCall("{call sp_acreditar(?, ?, ?)}")) {
    cs.setLong(1, cuentaId);
    cs.setBigDecimal(2, importe);
    cs.registerOutParameter(3, Types.VARCHAR);
    cs.execute();
    String resultado = cs.getString(3);
}

// Spring: SimpleJdbcCall
SimpleJdbcCall call = new SimpleJdbcCall(jdbcTemplate)
    .withProcedureName("SP_ACREDITAR")
    .declareParameters(
        new SqlParameter("P_CUENTA_ID", Types.NUMERIC),
        new SqlParameter("P_IMPORTE", Types.NUMERIC),
        new SqlParameter("P_RESULTADO", Types.VARCHAR));

Map<String, Object> out = call.execute(Map.of("P_CUENTA_ID", id, "P_IMPORTE", importe));
String resultado = (String) out.get("P_RESULTADO");

```

Diccionario de datos útil (Oracle)

```

SELECT * FROM user_tables;
SELECT column_name, data_type, nullable FROM user_tab_columns WHERE table_name = 'CUENTA';
SELECT * FROM user_constraints WHERE table_name = 'CUENTA';
SELECT * FROM user_indexes WHERE table_name = 'CUENTA';
SELECT text FROM user_source WHERE name = 'SP_ACREDITAR' ORDER BY line;
SELECT * FROM user_errors WHERE name = 'SP_ACREDITAR'; -- después de compilar con errores

```